

MOVIEDIRECTOR.AI

Operator Setup Guide

Everything you must configure on your side
before mass onboarding and revenue collection

Date: 2026-07-14

Product: MovieDirector.ai (Grok-powered film studio)

Audience: Founders / operators configuring production

Status: App code is ship-ready — this PDF is your setup checklist

Contents

1. Purpose of this guide
2. Revenue model at a glance
3. Accounts you need to create
4. Full environment variable list
5. Setup A — Core product (generation)
6. Setup B — Stripe (membership MRR + usage)
7. Setup C — Email, Redis, newsletters
8. Setup D — Optional render worker
9. Vercel deployment steps
10. Health probe and smoke tests
11. Go-live order of operations
12. Security and ops notes

1. Purpose of this guide

The MovieDirector.ai application is already built end-to-end: user accounts, projects, Grok image/video generation, public feed, messaging, channels, membership plans, and credit-based usage billing. **Your job as operator** is to connect third-party services and paste secrets into Vercel (or your host).

When core keys are set, users can sign up, create films, generate with Grok, and publish. When Stripe is set, you collect membership fees (MRR) and pay-per-use credit packs. After deploy, verify with **GET /api/health**.

Golden rule

Never commit real API keys to Git. Use Vercel Environment Variables only. Share this PDF with anyone helping configure accounts.

2. Revenue model at a glance

Stream	What users pay	How it collects
Membership	Free / Creator \$39 / Pro \$99 / Studio \$299 per month	Stripe Checkout subscription
Usage credits	Image 8 cr; video 10 cr/sec; packs \$19 / \$79 / \$299	Stripe one-time Checkout
Channels	Creator-set prices (subscribe free in beta)	Future Stripe Connect

Signup grants **120 free starter credits**. Paid plans grant monthly credits on `invoice.paid`. Failed generations auto-refund credits. Out of credits returns HTTP 402 and the app sends the user to Billing.

3. Accounts you need to create

Tick each box as you complete it.

- ☐ MongoDB Atlas — database cluster + connection string
- ☐ xAI / Grok — API key for image + video generation
- ☐ Vercel — project linked to GitHub repo moviedirector-ai
- ☐ Vercel Blob — storage token for permanent media URLs
- ☐ Stripe — account in Test mode first, then Live
- ☐ Upstash Redis (recommended) — rate limiting at scale
- ☐ SendGrid (recommended) — password reset + welcome emails
- ☐ beehiiv (optional) — newsletter / audience sync
- ☐ Render.com (optional) — full MP4 stitch worker
- ☐ Custom domain (optional) — e.g. moviedirector.ai pointed to Vercel

4. Full environment variable list

4.1 Required for generation (core ready)

```
MONGODB_URI=mongodb+srv://USER:PASS@cluster.../moviedirector
JWT_SECRET=<long-random-string-32+-chars>
XAI_API_KEY=<from console.x.ai>
XAI_VIDEO_MODEL=grok-imagine-video
XAI_IMAGE_MODEL=grok-imagine-image
BLOB_READ_WRITE_TOKEN=vercel_blob_rw...
NEXT_PUBLIC_APP_URL=https://YOUR_PRODUCTION_DOMAIN
```

4.2 Required for payments (full ready / MRR)

```
STRIPE_SECRET_KEY=sk_test... # then sk_live... for production
STRIPE_WEBHOOK_SECRET=whsec...
STRIPE_PRICE_CREATOR=price... # $39/mo recurring
STRIPE_PRICE_PRO=price... # $99/mo recurring
STRIPE_PRICE_STUDIO=price... # $299/mo recurring
STRIPE_PRICE_PACK_200=price... # $19 one-time 200 credits
STRIPE_PRICE_PACK_1000=price... # $79 one-time 1000 credits
STRIPE_PRICE_PACK_5000=price... # $299 one-time 5000 credits
```

4.3 Recommended for scale

```
UPSTASH_REDIS_REST_URL=https://...upstash.io
UPSTASH_REDIS_REST_TOKEN=...
SENDGRID_API_KEY=SG...
SENDGRID_FROM_EMAIL=noreply@yourdomain.com
BEEHIIV_API_KEY=...
BEEHIIV_PUBLICATION_ID=pub...
```

4.4 Optional render worker

```
RENDER_WORKER_URL=https://your-service.onrender.com
RENDER_WORKER_SECRET=<shared-secret>
MOVIEDIRECTOR_API_URL=https://YOUR_PRODUCTION_DOMAIN
```

5. Setup A — Core product (generation)

5.1 MongoDB Atlas

- Create a free or paid cluster in MongoDB Atlas.
- Create database user + password; allow network access (0.0.0.0/0 for Vercel, or restrict carefully).
- Copy connection string; set database name to moviedirector (or your choice).
- Paste into Vercel as MONGODB_URI (Production + Preview).

5.2 JWT secret

- Generate a long random secret: `openssl rand -base64 48`
- Set `JWT_SECRET` in Vercel Production + Preview.
- Do not reuse a weak or publicly shared secret in production.

5.3 xAI / Grok API

- Open the xAI console and create an API key with image + video generation access.
- Set `XAI_API_KEY` on Vercel.
- Confirm models: `XAI_VIDEO_MODEL=grok-imagine-video` and `XAI_IMAGE_MODEL=grok-imagine-image` (update if xAI renames models).
- Fund the xAI account and set spend alerts — video is the main cost driver.

5.4 Vercel Blob

- In the Vercel project: Storage → Blob → create store if needed.
- Copy `BLOB_READ_WRITE_TOKEN` into environment variables.
- Blob stores permanent frame/clip URLs (raw Grok media URLs expire).

5.5 Public app URL

- Set `NEXT_PUBLIC_APP_URL` to your production URL (e.g. `https://moviedirector-psi.vercel.app` or custom domain).
- Used for emails, Open Graph metadata, and Stripe success/cancel redirects.

After Setup A

GET `/api/health` should report status `core_ready` (or `ready` if Stripe is also configured). Users can sign up, receive 120 credits, generate, and publish.

6. Setup B — Stripe payments (MRR + usage)

6.1 Products & prices to create in Stripe Dashboard

Start in **Test mode**. Mirror the same products in **Live mode** before public launch.

Product name	Type	Amount	Env var for Price ID
MovieDirector Creator	Recurring monthly	\$39	STRIPE_PRICE_CREATOR
MovieDirector Pro	Recurring monthly	\$99	STRIPE_PRICE_PRO
MovieDirector Studio	Recurring monthly	\$299	STRIPE_PRICE_STUDIO
Credit Pack 200	One-time	\$19	STRIPE_PRICE_PACK_200
Credit Pack 1000	One-time	\$79	STRIPE_PRICE_PACK_1000
Credit Pack 5000	One-time	\$299	STRIPE_PRICE_PACK_5000

After creating each price, copy the **Price ID** (starts with price_) into the matching Vercel environment variable. Product names can vary; Price IDs must match env exactly.

6.2 API keys

- Developers → API keys → Secret key → STRIPE_SECRET_KEY (sk_test_... then sk_live_...).
- Never put the secret key in client-side NEXT_PUBLIC_* variables.

6.3 Webhook endpoint

- Developers → Webhooks → Add endpoint.
- URL: `https://YOUR_DOMAIN/api/billing/webhook`
- Events to enable:
 - `checkout.session.completed`
 - `invoice.paid`
 - `invoice.payment_failed`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
- Copy Signing secret → STRIPE_WEBHOOK_SECRET (whsec_...).
- Local testing: `stripe listen --forward-to localhost:3000/api/billing/webhook`

6.4 Customer Portal

- Settings → Billing → Customer portal → activate.
- Allow: cancel subscription, update payment method, view invoices.
- App opens portal via POST `/api/billing/portal`.

6.5 What Stripe automates in our code

Event / action	App behavior
Checkout subscription	Sets plan, Stripe IDs, grants monthly credits
Checkout credit pack	Adds pack credits to wallet

Event / action	App behavior
invoice.paid (renewal)	Grants monthly credits again
subscription deleted	Plan falls back to free
payment failed	Marks planStatus past_due

7. Setup C — Email, rate limits, newsletters

7.1 SendGrid (transactional email)

- Create SendGrid account; verify sender domain or single sender.
- Create API key with Mail Send permission → SENDGRID_API_KEY.
- Set SENDGRID_FROM_EMAIL to a verified address (e.g. noreply@yourdomain.com).
- Powers: welcome email, password reset, channel receipts.

7.2 Upstash Redis (rate limiting)

- Create Redis database in Upstash.
- Copy REST URL + token → UPSTASH_REDIS_REST_URL / UPSTASH_REDIS_REST_TOKEN.
- Without Upstash, app falls back to in-memory limits (not ideal multi-instance).

7.3 beehiiv (optional marketing list)

- Create publication; get API key + publication ID.
- Set BEEHIIV_API_KEY and BEEHIIV_PUBLICATION_ID.
- Signup with email syncs to list (best-effort, non-blocking).

8. Setup D — Optional render worker

Without a worker, users still get a professional ZIP export (clips + manifest + ffmpeg instructions). With a worker, one-click stitched MP4 is possible.

- Deploy the worker service under /worker (or your stitch service) on Render.
- Set RENDER_WORKER_URL and shared RENDER_WORKER_SECRET on both worker and Vercel.
- Set MOVIEDIRECTOR_API_URL to production app URL.

9. Vercel deployment steps

- Confirm GitHub repo is connected (moviedirector-ai) and Vercel project is moviedirector.
- Project Settings → Environment Variables → add every variable from sections 4.1–4.3 (and 4.4 if using worker). Scope: Production (and Preview if you test on previews).
- Redeploy after saving env vars (Deployments → Redeploy, or push a commit).
- Domains: attach custom domain if ready; update NEXT_PUBLIC_APP_URL + Stripe webhook URL.
- Stripe webhook must point at the final production domain (not a random preview URL).

9.1 Suggested Production vs Preview

Variable class	Production	Preview
Mongo / JWT / Blob	Production cluster	Can share or separate DB
XAI_API_KEY	Live key + spend caps	Same or lower-spend key
Stripe	sk_live_ + live prices	sk_test_ + test prices
Webhook	Production domain	Stripe CLI or test endpoint

10. Readiness probe & smoke tests

10.1 Health endpoint

```
GET https://YOUR_DOMAIN/api/health
```

status **"ready"** = core + Stripe fully configured. **"core_ready"** = generation OK, payments still pending. **"not_ready"** = missing Mongo, JWT, XAI, or Blob.

10.2 Pre-launch smoke checklist

- [] npm run build succeeds locally or on Vercel
- [] /api/health returns expected flags
- [] Signup grants 120 credits (check Billing or header credit chip)
- [] With XAI key: generate one frame, then one video; credits debit
- [] Force a failure if possible — credits refund
- [] Stripe Test: subscribe Creator → plan + monthly credits applied
- [] Stripe Test: buy a credit pack → balance increases
- [] Billing portal opens for a customer with stripeCustomerId
- [] Publish a film to the public feed as a new user

[] Confirm rate limits return 429 under rapid abuse

11. Go-live order of operations

Do these in order to minimize risk and double work.

1. Complete Setup A (Mongo, JWT, xAI, Blob, APP_URL) on Vercel Production.
2. Deploy and verify `/api/health` → `core_ready`.
3. Internal dogfood: `signup` → `create project` → `generate` → `publish`.
4. Complete Stripe Test products, prices, webhook, portal (Setup B in Test mode).
5. Verify Test checkout for membership + credit pack.
6. Configure SendGrid + Upstash; retest password reset and signup email.
7. Duplicate Stripe products/prices in Live mode; swap to `sk_live_` + live price IDs + live webhook.
8. Confirm `/api/health` → `ready`.
9. Soft launch to a small cohort; watch xAI spend and Stripe dashboard.
10. Open marketing / mass onboarding when metrics and support load are stable.

12. Security & ops notes

- Rotate any key that may have been shared in chat or committed by mistake.
- Restrict MongoDB user privileges to the app database only.
- Monitor xAI usage daily for the first two weeks (video COGS).
- Stripe disputes: keep clear plan descriptions and credit purchase receipts.
- Backup: enable Atlas backups; Vercel Blob assets are user media.
- Support path: password reset requires SendGrid; otherwise users cannot reset by email.
- Legal: add Terms + Privacy + refund policy links before heavy paid ads (product code does not replace legal).

Quick reference — money APIs

Method	Path	Purpose
GET	/api/billing/plans	Public plan + pack catalog
GET	/api/billing/account	User balance, plan, usage log
POST	/api/billing/checkout	Start Stripe Checkout
POST	/api/billing/portal	Open customer portal
POST	/api/billing/webhook	Stripe events (server only)
GET	/api/health	Launch readiness flags

End of Operator Setup Guide

Also in repo: LAUNCH.md · .env.example · SESSION_STATUS.md
When core + Stripe env are set, you are ready to onboard mass users and collect revenue.